

Redes de Comunicaciones y Cómputo Distribuido

Resumen teórico, explicado fácil

2do cuatrimestre 2026 — teóricas 03 a Transporte/TCP

Cómo leer esto

Cada tema arranca con una **idea intuitiva** (caja azul), sigue con lo que hay que saber y cierra con lo que **conviene tener a mano para la pregunta** (caja verde). Lo marcado con **libro** no está en las diapos: sale de la bibliografía que citan las clases (Peterson & Davie, *A Systems Approach*, 5ta; Tanenbaum & Wetherall, 5ta).

0. El mapa: capas

La idea

Mandar datos por una red es como mandar una carta por correo: vos escribís la carta (aplicación), la metés en un sobre con dirección (transporte/red), el correo la mete en un camión (enlace) que anda por una ruta (físico). Cada nivel sólo se preocupa por su parte y *usa* al de abajo sin saber cómo funciona.

- Cada capa **encapsula**: le agrega su propio *header* (y a veces *trailer*) al paquete de la capa de arriba.
- **OSI** tiene 7 capas (física, enlace, red, transporte, sesión, presentación, aplicación). **TCP/IP** (Internet) usa en la práctica: enlace/física, red (IP), transporte (TCP/UDP), aplicación.
- La capa N de un host “habla” con la capa N del otro mediante un **protocolo** (comunicación *par a par*).
- Los **routers** sólo llegan hasta la capa 3 (red). Transporte y aplicación son **end-to-end**: sólo existen en los hosts de las puntas.

1 Nivel Físico I: señales

La idea

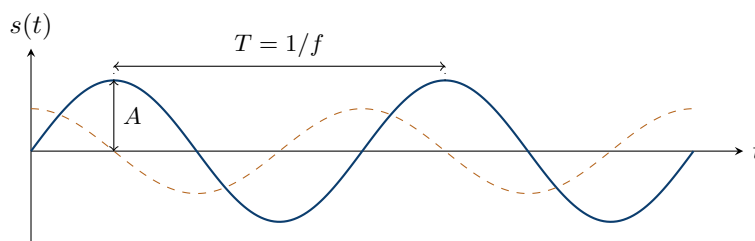
El nivel físico responde: *¿cómo convierto bits en algo que viaje por un cable, una fibra o el aire?* La respuesta es siempre una **señal** (voltaje, luz, onda de radio) que varía en el tiempo. Todo el tema gira alrededor de una pregunta: **¿qué tan rápido puede cambiar esa señal sin que el medio la arruine?** De eso depende cuántos bits por segundo puedo mandar.

1.1 Ondas: lo básico

Las señales viajan como **ondas electromagnéticas**. En el vacío van a la velocidad de la luz, $c = 3 \cdot 10^8$ m/s; en otros medios van más lento (en un cable de cobre UTP, $\approx 0,69c$; en fibra, $\approx \frac{2}{3}c$).

La onda más simple es la **senoidal**:

$$s(t) = A \cdot \sin(2\pi ft + \varphi)$$



En azul una senoide; en naranja otra con *menor amplitud* y *otra fase* (corrida en el tiempo).

- **Amplitud** A : qué tan “alta” es (volts, potencia).
- **Frecuencia** f : cuántas veces por segundo se repite (Hz). **Período** $T = 1/f$: cuánto dura un ciclo.
- **Fase** φ : dónde arranca el ciclo (un corrimiento en el tiempo).
- **Longitud de onda** $\lambda = v/f$: cuántos metros ocupa un ciclo mientras viaja.

Estas tres cosas (A , f , φ) son justamente las “perillas” que después vamos a mover para meter bits en la señal (modulación).

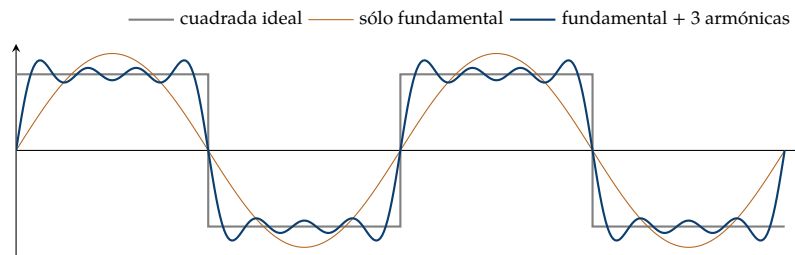
1.2 Fourier: toda señal es una suma de senoides

La idea: un acorde

Un acorde de guitarra suena como un solo sonido, pero en realidad es la suma de varias notas. Fourier dice lo mismo de cualquier señal periódica: es una **suma de senoides** de frecuencias $f_0, 2f_0, 3f_0, \dots$ (la fundamental y sus **armónicas**), cada una con su amplitud y fase.

El ejemplo clave es la **onda cuadrada**, que es lo que “queríamos” mandar para representar bits:

$$\text{cuadrada}(t) = \frac{4}{\pi} \left(\sin(\omega t) + \frac{1}{3} \sin(3\omega t) + \frac{1}{5} \sin(5\omega t) + \frac{1}{7} \sin(7\omega t) + \dots \right)$$



Con pocas armónicas la cuadrada queda “redondeada”; con más, se parece cada vez más. Una cuadrada *perfecta* (cambio instantáneo de 0 a 1) necesita **infinitas** armónicas, o sea, infinitas frecuencias.

Esto permite mirar una señal de dos formas: en el **dominio del tiempo** (cómo varía) o en el **dominio de la frecuencia** (su *espectro*: qué frecuencias tiene y con cuánta amplitud).

1.3 Ancho de banda: el canal es un filtro

La idea

Ningún medio deja pasar todas las frecuencias igual: las muy altas se atenúan (se “pierden”). El canal se comporta como un **filtro**. El **ancho de banda** B es el rango de frecuencias que pasa razonablemente bien.

- Si mando una cuadrada por un canal con poco B , se pierden las armónicas altas y llega una señal redondeada. Si los bits van muy rápido, se “mezclan” y el receptor ya no los distingue.
- Por eso **el ancho de banda limita la velocidad**: más B permite que la señal cambie más rápido, y entonces más bits por segundo.
- Formalmente, la frecuencia de corte es donde la potencia cae a la mitad: -3 dB.
- **Decibeles**: $\text{dB} = 10 \log_{10}(P_1/P_2)$. Sirven para comparar potencias con números chicos: 3 dB $\approx$ el doble, 10 dB $= 10$ veces, 20 dB $= 100$ veces, 30 dB $= 1000$ veces.

1.4 Delay: cuánto tarda en llegar un mensaje

La idea: una caravana de autos

Mandar una trama es como mandar una caravana de autos por una ruta. El **tiempo de transmisión** es cuánto tardan todos los autos en *salir* por el peaje (depende de cuántos autos hay y de qué tan rápido pasa cada uno). El **tiempo de propagación** es cuánto tarda un auto en *recorrer* la ruta (depende de la distancia). Si hay cola en el peaje, se suma el **encolamiento**.

$$\text{Delay} = T_{\text{prop}} + T_{\text{trans}} + T_{\text{encol}} + T_{\text{proc}}$$

- $T_{\text{prop}} = \frac{\text{distancia}}{v}$: pesa en enlaces **largos** (satélite, intercontinentales).
- $T_{\text{trans}} = \frac{\text{tamaño trama}}{\text{velocidad de TX}}$: pesa en enlaces **lentos** o tramas **grandes**.
- T_{encol} : lo que se espera en buffers de routers; depende de la congestión (de 0 a enorme).
- T_{proc} : lo que tarda el nodo en procesar; microsegundos, casi siempre despreciable.

Ejemplo: 1500 bytes por un enlace de 10 Mbps, 2000 km de cobre ($2 \cdot 10^8$ m/s): $T_{\text{trans}} = 12000/10^7 = 1,2$ ms, $T_{\text{prop}} = 2 \cdot 10^6/2 \cdot 10^8 = 10$ ms. Acá manda la propagación.

2 Teoría de la Información (Shannon, 1948)

La idea

Shannon se preguntó: *¿qué es la información y cuánto puedo mandar por un canal con ruido?* Su gran idea fue ignorar el **significado** del mensaje y mirar sólo su **estadística**. Informa más lo que **sorprende**: “mañana sale el sol” no informa nada; “mañana nieva en Buenos Aires” informa mucho.

Dos preguntas, dos teoremas:

1. **Codificación de fuente**: ¿cuánto puedo *comprimir* un mensaje sin perder nada? (límite de la eficiencia)
2. **Codificación de canal**: ¿qué tan rápido puedo mandar *sin errores* por un canal ruidoso? (límite de la confiabilidad)

2.1 Información y entropía

- **Información** de un símbolo con probabilidad p : $I = \log_2 \frac{1}{p}$ bits. Si $p = 1$ (seguro), $I = 0$. Cuanto menos probable, más informa.
- **Fuente de memoria nula**: cada símbolo es independiente de los anteriores.
- **Entropía**: la información *promedio* por símbolo, o sea la “sorpresa promedio”:

$$H(S) = \sum_i p_i \log_2 \frac{1}{p_i}$$

- Propiedades: $0 \leq H(S) \leq \log_2 n$. Vale 0 si un símbolo es seguro, y es máxima cuando todos son equiprobables (no se puede adivinar nada). Una moneda justa tiene $H = 1$ bit; una cargada, menos.

2.2 Codificación: comprimir

La idea: el código Morse

En Morse la E (la letra más común) es un solo punto, y las letras raras son largas. Asignar **códigos cortos a lo frecuente** achica el mensaje promedio.

- Un código útil tiene que ser **unívocamente decodificable**; mejor aún, **instantáneo** (se decodifica sin mirar adelante).
- **Condición de prefijo**: es instantáneo *sii* ninguna palabra es prefijo de otra.
- **Longitud media**: $L = \sum p_i L_i$. **Teorema de fuente**: $L \geq H$. No se puede comprimir por debajo de la entropía.
- **Huffman**: juntar repetidamente los dos símbolos menos probables en un árbol (izquierda 0, derecha 1). Da un código prefijo óptimo. Ej.: “MI MAMA ME MIMA” ocupa 33 bits con Huffman contra 120 en ASCII.

2.3 El canal real: ruido y perturbaciones

- **Atenuación**: la señal se debilita con la distancia, y más a frecuencias altas. Se compensa con amplificadores y ecualización.
- **Distorsión de retardo** (medios guiados): cada frecuencia viaja a distinta velocidad y las componentes llegan desfasadas.
- **Ruido**: señal que se suma y no es mía.
 - **Térmico**: agitación de los electrones, inevitable y parejo en todas las frecuencias (“blanco”): $N = kTB$. **A más ancho de banda, más ruido.**
 - **Intermodulación**: el medio no es lineal y aparecen frecuencias suma/diferencia.
 - **Diafonía** (crosstalk): un cable “se escucha” en el de al lado.
 - **Impulsivo**: picos cortos y grandes (una tormenta). Es el peor para los datos.
- En digital, el ruido se traduce en bits cambiados: **BER** (Bit Error Rate).

2.4 ¿Cuántos bits por segundo entran? Nyquist y Shannon

La idea

Imaginá que codificás bits con **niveles de voltaje**. Hay dos límites: (1) qué tan **seguido** puedo cambiar de nivel, que lo fija el ancho de banda, y (2) cuántos niveles **distingo** sin confundirlos, que lo fija el ruido.

- **Nyquist** (canal *sin* ruido): en un canal de ancho B la señal puede cambiar a lo sumo $2B$ veces por segundo. Con M niveles cada cambio lleva $\log_2 M$ bits:

$$C = 2B \log_2 M \text{ bps}$$

Sin ruido, con más niveles tendría velocidad infinita.

- **Baudio** = cambios de señal (símbolos) por segundo. $\text{bps} = \text{baudios} \times \text{bits por símbolo}$.
- **Shannon** (canal *con* ruido): el ruido no deja distinguir niveles muy juntos. El techo absoluto es

$$C_{\max} = B \log_2(1 + \text{SNR}) \quad \text{SNR} = \frac{P_{\text{señal}}}{P_{\text{ruido}}}, \quad \text{SNR}_{\text{dB}} = 10 \log_{10} \text{SNR}$$

Ej.: línea telefónica, $B = 3000 \text{ Hz}$ y $\text{SNR} = 30 \text{ dB}$ ($= 1000$), da $C \approx 3000 \cdot \log_2 1001 \approx 30 \text{ kbps}$.

- Teorema de canal: si transmito a una tasa $R < C$, *existe* un código que hace el error tan chico como quiera; si $R > C$, no hay forma. Shannon no dice *cómo* construir ese código.
- Subir B no ayuda para siempre, porque también sube el ruido ($N = N_0 B$): la capacidad se satura. Subir la potencia aumenta las no linealidades.

Para recordar

Nyquist dice *cuántos niveles harían falta*; Shannon pone el *techo* por el ruido. La capacidad real es la menor de las dos. Entropía = sorpresa promedio = mínimo de bits por símbolo para comprimir.

3 Nivel Físico II: medios, multiplexación y modulación

3.1 Medios de transmisión

- **Guiados** (la señal va “encerrada”):
 - **Par trenzado**: dos cables de cobre enroscados. El trenzado cancela interferencias y diafonía. Barato; es el típico cable de red.
 - **Coaxial**: un conductor central blindado. Usado en la TV por cable (CATV).
 - **Fibra óptica**: luz que rebota adentro de un vidrio muy puro (*reflexión total interna*). Casi no se atenúa y es inmune a interferencias eléctricas. *Monomodo* (un solo camino de luz, para larga distancia) vs *multimodo* (más barata, corta distancia).
- **No guiados** (por el aire): radio, microondas, infrarrojo, láser, satélite, Li-Fi (datos codificados en el parpadeo de LEDs).

3.2 Multiplexar: compartir un medio

La idea: una autopista

Tengo *un* cable caro y muchas conversaciones. **FDM**: cada una va por su carril (su banda de frecuencias), todas al mismo tiempo, como las radios FM. **TDM**: todas usan la autopista entera pero por turnos (round-robin).

- **FDM**: cada usuario tiene una porción de frecuencias *todo el tiempo*. Requiere electrónica analógica.
- **TDM**: cada usuario usa *todo* el ancho de banda en su turno. Sólo sirve para señales digitales, pero es simple y barato. Es lo que usa la telefonía digital.
- **WDM**: FDM en fibra, con un “color” (longitud de onda) por canal.
- **Multiplexación estadística** (redes de paquetes): TDM *bajo demanda*. Nadie tiene turno fijo: el que tiene datos los manda y, si el enlace está ocupado, espera en una cola. Si la cola se llena hay **congestión**.

Red telefónica (PSTN): *local loop* analógico (par trenzado hasta tu casa), troncales digitales (fibra), centrales de conmutación. Es **conmutación de circuitos**: durante la llamada tenés un camino reservado para vos.

3.3 Pasar de analógico a digital: PCM

La idea: fotos de una señal

Para digitalizar la voz le saco “fotos” (muestras) cada tanto y anoto cada una como un número. ¿Cada cuánto? Si saco pocas fotos pierdo detalle (como las ruedas de los autos que en las películas parecen girar al revés).

- **Teorema de muestreo (Nyquist)**: para reconstruir una señal cuya frecuencia máxima es f_m , hay que muestrear a $f_s > 2f_m$. CD: 44,1 kHz sirve para audio de hasta 22 kHz. Voz telefónica (hasta 4 kHz): 8000 muestras/s.
- Dos pasos: **muestreo** (da pulsos de altura variable, PAM) y **cuantificación** (redondear cada pulso a un número de n bits). El redondeo introduce *error de cuantificación*.
- **PCM** de voz: 8000 muestras/s $\times$ 8 bits = **64 kbps** por canal, una muestra cada 125 μ s.
- **T1** (EE.UU.): 24 canales $\times$ 8 bits + 1 bit de sincronismo = 193 bits cada 125 μ s = 1,544 Mbps. **E1** (Europa): 32 canales, 2,048 Mbps.
- **Códec** (analógico $\rightarrow$ digital) $\neq$ **Módem** (digital $\rightarrow$ analógico, para mandar bits por un medio analógico).

3.4 Modulación: meter bits en una onda

La idea

Tomo una senoide “portadora” y muevo una de sus perillas según el bit que quiero mandar.

- **ASK**: cambio la *amplitud* (fuerte = 1, débil o nada = 0).
- **FSK**: cambio la *frecuencia* (aguda = 1, grave = 0).
- **PSK**: cambio la *fase* (sin corrimiento = 0, invertida = 1).
- **Multinivel**: si uso 4 fases distintas, cada símbolo lleva 2 bits (QPSK). En general, con S símbolos posibles:

$$V_t = V_m \cdot \log_2 S \quad (\text{bps} = \text{baudios} \times \text{bits por símbolo})$$

- **QAM**: combino amplitud y fase. Cada combinación es un punto en un dibujo llamado **constelación**. 16-QAM lleva 4 bits por símbolo, 64-QAM 6 y 256-QAM 8.

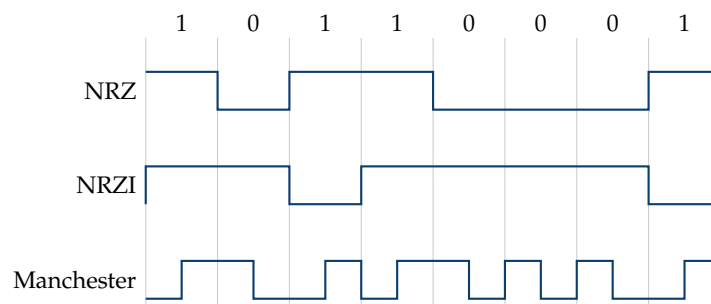
La otra cara de la moneda

Más puntos en la constelación implica puntos más juntos, y entonces el ruido los confunde más fácil: **más BER** con la misma SNR. Por eso no puedo subir niveles para siempre. Es Shannon otra vez.

3.5 Codificación de línea: bits como voltajes

La idea

Si mando bits “crudos” como alto/bajo, una tira larga de ceros es una línea plana, y el receptor pierde la cuenta de cuántos bits pasaron (**pierde el reloj**). Las codificaciones de línea meten **transiciones** para que el receptor se sincronice.



- **NRZ**: 1 = alto, 0 = bajo. Simple, pero con muchos bits iguales seguidos no hay transiciones y se pierde el sincronismo.
- **NRZI**: un 1 es una *transición*, un 0 es quedarse igual. Resuelve muchos 1 seguidos, no muchos 0.
- **Manchester**: siempre hay una transición *en el medio* de cada bit (subir = 1, bajar = 0). El reloj viene incluido, pero la señal cambia el doble de rápido: **baud rate** = $2 \times$ **bit rate**, es decir 50 % de eficiencia. Lo usa Ethernet de 10 Mbps.

- **Manchester diferencial:** 0 = hay transición al *inicio* del bit, 1 = no hay (en el medio siempre hay).
- **libro 4B/5B:** cada 4 bits se mandan como 5, elegidos para que nunca haya muchos ceros seguidos, y después NRZI. Eficiencia del 80 %.

4 Nivel de Enlace: punto a punto

La idea

El nivel físico me da un “caño” que lleva bits en orden pero **a veces se equivoca**. El nivel de enlace arma **tramas** (paquetitos) y se encarga de detectar errores y, si hace falta, de retransmitir.

4.1 Framing

¿Dónde empieza y termina cada trama dentro de un chorro de bits? Hay tres opciones: largo fijo, largo escrito en el header, o **delimitadores** (una secuencia especial, como en PPP 01111110). Si esa secuencia aparece en los datos se hace **bit stuffing**: **libro** después de cinco 1 seguidos se inserta un 0, y el receptor lo saca.

Tipos de servicio: sin conexión y sin ACK / sin conexión con ACK / orientado a conexión.

4.2 Detectar y corregir errores

La idea

Agrego bits **redundantes** para que un error “se note”: m bits de datos + r de redundancia forman una *codeword* de n bits. Si las codewords válidas están “lejos” entre sí, cambiar pocos bits cae en algo inválido.

- **Distancia de Hamming:** cuántos bits difieren entre dos palabras. d es la mínima entre todas las codewords del código.
- Para **detectar** e errores: $d \geq e + 1$. Para **corregir** e errores: $d \geq 2e + 1$ (la palabra recibida tiene que quedar más cerca de la original que de cualquier otra).
- **libro** En la práctica: paridad, *checksum* de Internet (suma en complemento a 1, lo usan IP/TCP/UDP) y **CRC** (división de polinomios, muy bueno con ráfagas de errores; lo usa Ethernet).

4.3 Transmisión confiable: Stop & Wait y ventana deslizante

Stop & Wait: mando una trama y espero su **ACK**; si vence un *timeout*, la reenvío.

- Si se pierde el ACK, reenvío y el receptor recibe un duplicado (**reencarnación**). Por eso se **numeran** las tramas; en S&W alcanza con 2 números (0 y 1).
- Problema: me paso casi todo el tiempo *esperando*. **Eficiencia** $\eta = T_{tx}/RTT$.

La idea: llenar el caño

El cable es un caño largo. Si mando una trama y espero, el caño está casi vacío. El **producto delay × ancho de banda** (**capacidad de volumen**, $C_{vol} = V_{tx} \cdot \text{Delay}$) dice cuántos bits “entran” en el caño. Para aprovecharlo tengo que mandar tramas hasta que vuelva el primer ACK, es decir durante un RTT.

Sliding window: mando varias tramas sin esperar.

- Ventana de emisión: $SWS = \frac{V_{tx} \cdot RTT}{|\text{Frame}|}$ tramas. Puedo mandar si $\text{ÚltimoEnviado} \leq \text{ÚltimoReconocido} + SWS$.
- **ACK acumulativo** (Go-Back-N): “recibí todo hasta n ”. El receptor no guarda tramas fuera de orden ($RWS = 1$); si se pierde una, se reenvía desde ahí.
- **ACK selectivo** (Selective Repeat): el receptor guarda las fuera de orden ($RWS = SWS$) y sólo se reenvía lo perdido.
- Para no confundir reencarnaciones: **cantidad de números de secuencia** $\geq SWS + RWS$.

Para recordar

$d \geq e + 1$ detecta, $d \geq 2e + 1$ corrige. S&W es ineficiente con RTT grande; la ventana tiene que cubrir $\text{delay} \times \text{BW}$; $\#seq \geq SWS + RWS$.

5 Nivel de Enlace: medios compartidos (MAC)

La idea: una mesa de gente charlando

Muchos hosts comparten un mismo cable (o el aire). Si dos hablan a la vez, no se entiende nada (**colisión**). No hay un moderador: cada uno tiene que seguir reglas de cortesía. Esas reglas son el **protocolo MAC** (Medium Access Control).

Objetivos: maximizar las transmisiones exitosas y que todos tengan oportunidades parecidas (*fairness*).

5.1 De ALOHA a Ethernet

- **ALOHA** (Hawaii, 1970): “hablo cuando quiero; si no me contestan, espero un rato al azar y repito”. Aprovecha como mucho el 18 % del canal. **libro** Slotted ALOHA (hablar sólo al inicio de un turno): 37 %.
- **CSMA** (*Carrier Sense*): “escucho antes de hablar”. Si está ocupado: *1-persistente* (espero a que se libere y hablo enseguida, como Ethernet) o *p-persistente* (hablo con probabilidad p).
- **CSMA/CD** (Ethernet 802.3): además, *mientras hablo escucho*. Si detecto una colisión corto, mando una señal de *jam* y espero.

Por qué Ethernet tiene trama mínima

Para detectar una colisión tengo que **seguir transmitiendo** hasta que la colisión pueda volver a mí. En el peor caso (A y B en los extremos), eso tarda $2 \cdot T_{prop}$. Si la trama es más corta, termino de mandar antes de enterarme y creo que salió bien. Ethernet 10 Mbps: trama mínima de 512 bits (64 bytes). Esto limita el largo de la red (2500 m, 4 repetidores).

- **Exponential backoff**: después de la k -ésima colisión, elijo al azar un número de slots entre 0 y $2^k - 1$. Cuantas más colisiones, más me abro (**libro** tope $k = 10$, abandono a los 16 intentos).
- Trama Ethernet: preámbulo (8 B) | destino (6) | origen (6) | tipo/largo (2) | datos (46–1500) | CRC (4).
- CSMA escala mal: con mucha carga, colisiona todo el tiempo.
- **802.2 LLC**: subcapa encima de las distintas MAC (802.3, 802.11, ...) que las unifica y ofrece los 3 tipos de servicio.

5.2 Crecer: bridges, switches, STP y VLANs

- Dispositivos por capa: **físico**: repetidores y hubs (repiten todo); **enlace**: bridges y switches (miran la MAC); **red**: routers.
- **Dominio de colisión**: quiénes pueden chocar entre sí; lo corta un switch/bridge (un hub no). **Dominio de broadcast**: a quiénes llega un broadcast; lo corta un router (o una VLAN).
- **Learning bridge**: mira la MAC *origen* de cada trama y anota “esta MAC está en este puerto”. Después reenvía sólo por donde hace falta. Si no conoce el destino, **inunda** (manda por todos lados). Los broadcast siempre se inundan.

Spanning Tree: sacar los ciclos

Si hay caminos redundantes entre bridges, una trama inundada da vueltas **para siempre**. STP apaga algunos puertos para que la red quede como un **árbol** (sin ciclos), pero si se cae un enlace los reactiva.

1. Se elige un **root** (el de menor ID).
2. Cada switch calcula su distancia al root. Su **root port** es el puerto que más cerca lo deja del root.
3. En cada LAN, el switch más cercano al root es el que reenvía por ella: su puerto es el **designated port**.
4. Todo lo demás queda **bloqueado**. Se intercambian **BPDUs** periódicamente (“soy el 3, creo que el root es el 2, estoy a distancia 1”).

“El broadcast no escala”: una LAN gigante tiene broadcasts por todos lados. Las **VLANs** parten una LAN física en varias *lógicas* (cada una su propio dominio de broadcast). Entre switches se usa **trunking 802.1Q**: se agrega un tag de 4 bytes con el ID de VLAN.

5.3 Wireless (802.11)

- Más ruido y errores, batería limitada y cualquiera puede escuchar (hay que cifrar). Bandas licenciadas (ENACOM) vs **no licenciadas** (ISM: potencia limitada y mucha interferencia), de ahí el *spread spectrum* (saltar de frecuencia, FHSS, idea de Hedy Lamarr).

- **Estación oculta:** A le habla a B; C no oye a A (está lejos), cree que el medio está libre, habla y arruina la recepción en B.
- **Estación expuesta:** B le habla a A; C oye a B y se calla, aunque podría hablarle a D sin molestar a nadie.
- En wireless no se puede escuchar mientras se transmite, y la colisión importa en el receptor, no en el emisor: **no hay CD**. Se usa **CSMA/CA**: escucho; si está libre espero un *IFS*; si está ocupado espero a que termine y además un *backoff* aleatorio (el contador se congela si el medio se ocupa). El receptor manda **ACK**; sin ACK asumo colisión y reintento. **libro** Opcional: **RTS/CTS** (pido permiso; todos los que oyen el “CTS” del receptor se callan), que ataca la estación oculta.

Para recordar

Trama mínima $\leftrightarrow 2T_{prop}$. Backoff exponencial. Switch corta colisión, router/VLAN corta broadcast. STP: root, root port, designated, bloqueado. Wireless: CA en vez de CD por oculta/expuesta.

6 Nivel de Red: datagramas vs circuitos virtuales

La idea

Un **switch** (en sentido amplio) recibe paquetes por un puerto y decide por cuál sacarlos. Hay dos filosofías: **datagramas** (cada paquete es una carta con la dirección completa, como el correo) y **circuitos virtuales** (primero “llamo” y armo un camino, y después todos los paquetes lo siguen, como una llamada telefónica).

	Datagramas (IP)	Circuito virtual (X.25, Frame Relay, ATM)
Setup	No hace falta: mando cuando quiero	Hay que armarlo (1 RTT) y después cerrarlo
Dirección	Cada paquete lleva la dirección completa	Sólo un número chico: el VCI
Estado en switches	Ninguno	Una entrada por circuito: (puerto, VCI) de entrada $\rightarrow$ (puerto, VCI) de salida
Ruteo	Cada paquete por su cuenta	Se elige al armar el circuito
Si cae un switch	Se rutea por otro lado	Se caen todos sus circuitos
QoS y congestión	Difícil	Fácil: se reservan recursos al armar

El VCI tiene significado *local* a cada enlace (se reescribe en cada salto: *label switching*). **PVC**: lo arma el administrador. **SVC**: se arma por señalización cuando se necesita.

7 Nivel de Red: IP

La idea

IP es el “idioma común” que permite unir redes distintas (Ethernet, WiFi, fibra...) en **una sola Internet**. Promete poco a propósito: **best effort**. Hace lo que puede, sin garantías. La inteligencia queda en los extremos (TCP).

- Servicio sin conexión: los paquetes se pueden **perder, desordenar, duplicar y demorar** sin límite.
- **Header IPv4** (20 bytes, hasta 60 con opciones), los campos que importan:
 - *Versión, HLen* (largo del header en palabras de 32 bits), *TOS/DiffServ-ECN* (calidad de servicio), *Longitud total* (máximo 65535).
 - *Ident, Flags (DF, MF), Offset*: para fragmentar.
 - **TTL**: se resta 1 en cada router y en 0 se descarta. Evita que un paquete dé vueltas para siempre.
 - *Protocol*: a quién le entrego los datos (1 ICMP, 6 TCP, 17 UDP, 89 OSPF).
 - *Checksum*: **sólo del header**. *Origen y destino*: 32 bits cada una.

7.1 Fragmentación

Cada tecnología de enlace tiene un tamaño máximo, el **MTU** (Ethernet 1500 B, FDDI 4500 B). Si un router tiene que mandar un paquete más grande que el MTU de la red siguiente, lo **parte**.

- Todos los fragmentos tienen el mismo *Ident*; todos menos el último llevan *MF*= 1; el *offset* se cuenta en unidades de 8 bytes.

- Se **reensambla sólo en el destino** (no en routers intermedios). Si se pierde un fragmento, se pierde todo el datagrama (IP no recupera nada).
- Toda red tiene que aceptar al menos 68 bytes. **libro** Hoy se evita fragmentar: se manda con DF= 1 y se descubre el MTU del camino (*Path MTU Discovery*).

7.2 Direcciones y forwarding

- Dirección de 32 bits (≈ 4300 millones), única y **jerárquica**: una parte identifica la *red* y otra el *host* (como calle y número). Notación: 157.92.x.x.
- Esquema original *classful* (clases A, B, C, de tamaños fijos): desperdiciaba direcciones y agrandaba las tablas. **libro** Se reemplazó por **CIDR**: prefijos de cualquier largo (/n), agregación de rutas, y se elige la ruta con el prefijo más largo que coincida (*longest prefix match*). Ej.: UBA = 157.92/16.
- **Direcciones privadas** (RFC 1918): 10/8, 172.16/12, 192.168/16. No salen a Internet (**libro** se usan detrás de NAT).
- **Forwarding**: si el destino está en una red a la que estoy conectado, se lo entrego directo; si no, busco en mi tabla el *next hop*; si no está, lo mando al **default router**. Un host común sólo sabe “¿es de mi red? directo : al default router”.
- **libro** Para entregar directo necesito la MAC del destino: **ARP**. Los errores (TTL vencido, destino inalcanzable) se informan con **ICMP** (así funcionan ping y traceroute).
- **IPv6**: 128 bits (8000:0000::...:CDEF). Header fijo y más simple (sin checksum y sin fragmentación en routers).

8 Nivel de Red: ruteo

La idea

Forwarding es mirar la tabla y sacar el paquete por un puerto (rápido, paquete a paquete). **Routing** es *armar* esa tabla: encontrar los caminos más baratos en un grafo que cambia, con un algoritmo distribuido.

- Ruteo *estático* (configurado a mano) vs *dinámico* (se adapta a fallas y carga).
- Internet es un conjunto de **Sistemas Autónomos** (AS): redes con una sola administración (un ISP, una universidad). Adentro de un AS se usa un protocolo **intradominio** (RIP, OSPF); entre AS, uno **interdominio** (BGP). Cada AS esconde sus detalles internos.

8.1 Vector de distancia (RIP)

La idea: rumores entre vecinos

Cada router le cuenta a sus **vecinos** “yo llego a X con costo tanto”. Cada uno se queda con la mejor opción: “si mi vecino llega a G en 12 y yo llego a él en 6, yo llego a G en 18 pasando por él”. Nadie conoce el mapa entero.

- Cada nodo tiene (Destino, Costo, NextHop) hacia *todos*. Manda **su tabla entera, sólo a sus vecinos**, periódicamente y cuando algo cambia.
- Es Bellman-Ford distribuido: $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$.
- Las buenas noticias viajan rápido y las malas lento: **conteo a infinito**. Si se cae un enlace, A y C se creen mutuamente (“C dice que llega en 2, entonces yo llego en 3”) y el costo sube de a poco sin fin.
- Parches: **infinito = 16**; **split horizon** (no le anuncio una ruta al vecino del que la aprendí); **poison reverse** (se la anuncio con costo ∞). Sólo arreglan ciclos de 2 nodos.
- **RIP**: costo = cantidad de saltos (1 a 15; 16 = ∞ , así que sirve sólo para redes chicas). Corre sobre **UDP**, puerto 520.

8.2 Estado de enlace (OSPF)

La idea: cada uno publica su pedacito de mapa

Cada router le cuenta a **toda la red** sólo *quiénes son sus vecinos y cuánto cuesta llegar a cada uno*. Con los pedacitos de todos, cada router arma el **mapa completo** y calcula los caminos con Dijkstra.

- Pasos: descubrir vecinos → medir costos → armar un **LSP** (ID, costos a vecinos, **número de secuencia**, **TTL**) → inundarlo → Dijkstra.
- **Inundación confiable**: guardo el LSP más nuevo de cada router (por número de secuencia), lo reenvío a todos

menos al que me lo dio, confirmo con ACK y descarto los viejos (TTL).

- **Dijkstra** (*forward search*): listas *Tentativo* y *Confirmado*. Confirmo el tentativo más barato y con su LSP actualizo a sus vecinos.
- **DV vs LS**: en DV cada uno le cuenta a sus vecinos todo lo que sabe (puede ser inestable). En LS cada uno le cuenta a todos lo de sus vecinos: es estable, converge rápido y todos ven lo mismo, pero usa más memoria y CPU.
- **Métricas** (ARPANET): primero largo de la cola; después retardo medido (oscilaba); finalmente utilización del enlace con un rango acotado.
- **OSPF**: LS abierto, directo sobre IP (protocolo 89). Mensajes autenticados, varios caminos de igual costo (balancea), métricas por tipo de servicio. Es **jerárquico**: se divide en *áreas* más un *área troncal*; los LSPs no salen de su área. Se **cambia optimalidad por escalabilidad** (esconder información puede dar caminos no óptimos). Mensajes: Hello, Database Description, LS Request, LS Update, LS Ack.

8.3 Entre sistemas autónomos: BGP

- **EGP** (viejo): suponía Internet con forma de árbol y sólo miraba alcanzabilidad.
- **BGP**: no exige árbol. Tipos de AS: **stub** (una sola conexión, sólo tráfico propio), **multihomed** (varias conexiones pero no deja pasar tráfico ajeno), **transit** (lleva tráfico propio y de terceros).
- Cada AS tiene un *speaker* BGP que anuncia “estas redes se alcanzan por el camino AS1, AS2, ...”. Se anuncia el **camino completo de AS** (**libro** *path vector*: así se detectan loops y cada AS aplica sus **políticas** comerciales, no sólo “el más corto”). Las rutas se pueden retirar. **libro** Va sobre TCP (puerto 179).
- **NAPs/IXPs**: puntos donde muchas redes intercambian tráfico directamente (en Argentina, CABASE). Abaratan y mejoran la calidad, porque el tráfico local no sale al exterior.

Para recordar

DV: tabla entera a vecinos, Bellman-Ford, conteo a infinito (16, split horizon, poison reverse), RIP. LS: vecinos a todos, Dijkstra, estable, OSPF jerárquico (optimalidad por escala). BGP: entre AS, caminos de AS y políticas.

9 Nivel de Transporte: UDP y TCP

La idea

IP lleva paquetes de *host* a *host*, sin garantías. Transporte lleva datos de *proceso* a *proceso* (con **puertos**), y TCP además construye, sobre esa red poco confiable, una **conexión confiable**: todo llega, en orden y una sola vez.

¿Por qué no alcanza la ventana del nivel de enlace? Porque acá la otra punta puede estar en cualquier lado: el RTT es distinto y cambia (timeouts adaptativos), pueden aparecer paquetes muy viejos, el receptor puede ser lento (control de flujo) y la red se puede congestionar (control de congestión).

9.1 UDP

Casi nada: agrega **puertos** (para saber a qué proceso va) y un checksum opcional. **libro** Header de 8 bytes. Sin conexión y sin garantías. Se usa cuando importa más la rapidez que la confiabilidad: audio/video en vivo (RTP), DNS, RIP.

9.2 TCP: qué ofrece

- **Orientado a conexión, full duplex, flujo de bytes**: la app escribe bytes, TCP los corta en segmentos, la app del otro lado lee bytes (no se respetan los “mensajes”).
- **Confiable**: números de secuencia, ACKs, checksum y retransmisión por timeout.
- **Control de flujo**: no ahogar al *receptor*. **Control de congestión**: no ahogar a la *red* (se ve en otra clase).
- Una conexión se identifica por la **4-tupla** (IP origen, puerto origen, IP destino, puerto destino).
- **MSS**: máximo de datos por segmento (default 536; en Ethernet $1500 - 20 - 20 = 1460$). Se manda un segmento cuando junto un MSS, cuando vence un timer, o cuando la app hace *push*.
- **Header** (20 bytes + opciones): puertos; **SequenceNum** (número del *primer byte* que va en el segmento); **Acknowledgment** (el *próximo byte que espero*, acumulativo); flags SYN, ACK, FIN, RST, PSH, URG; **Advertised-Window** (16 bits); checksum (incluye un pseudo-header con las IPs).

9.3 Abrir y cerrar

- **3-way handshake:** SYN → SYN+ACK → ACK. Cada lado elige su número de secuencia inicial (**libro** aleatorio, para no confundirse con conexiones viejas).
- **Cierre en 4 pasos:** FIN, ACK, FIN, ACK. Cada dirección se cierra por separado.
- **Estados:** CLOSED, LISTEN, SYN_SENT, SYN_RCVD, ESTABLISHED, FIN_WAIT_1, FIN_WAIT_2, CLOSE_WAIT, LAST_ACK, CLOSING y TIME_WAIT.
- **TIME_WAIT:** quien cierra primero espera $2 \times \text{MSL}$ (el doble de lo máximo que vive un paquete) antes de olvidar la conexión: así mueren los segmentos viejos y puede reenviar el último ACK si se perdió.

9.4 Ventana deslizante y control de flujo

La idea: el balde del receptor

El receptor tiene un balde (buffer). En cada ACK le avisa al emisor cuánto lugar le queda (**AdvertisedWindow**), y el emisor nunca tiene “en vuelo” más que eso.

$$\text{AdvertisedWindow} = \text{MaxRcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

$$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

- Emisor: $\text{LastByteAcked} \leq \text{LastByteSent} \leq \text{LastByteWritten}$. Receptor: $\text{LastByteRead} < \text{NextByteExpected} \leq \text{LastByteRcvd} + 1$ (si hay un hueco, NextByteExpected apunta al hueco).
- Retransmite estilo Go-Back-N con ACK acumulativo (**libro** hay una extensión SACK para ACKs selectivos).
- Si la ventana llega a 0, el emisor manda cada tanto un *window probe* de 1 byte (**persist timer**). Si no, el ACK que reabre la ventana se podría perder y los dos quedarían esperando para siempre.

9.5 ¿Cuánto esperar antes de retransmitir? (RTO)

La idea

Si espero poco, retransmito cosas que no se perdieron; si espero mucho, tardo en recuperarme. Y el RTT en Internet **varía muchísimo**. Solución: ir **estimando** el RTT y su variación, y poner el timeout un poco por encima.

- **Original:** $\text{EstRTT} = \alpha \text{EstRTT} + (1 - \alpha) \text{Sample}$ (promedio que “recuerda”), y $\text{timeout} = 2 \cdot \text{EstRTT}$.
- **Karn/Partridge:** si retransmití, no sé si el ACK es del original o de la copia, así que **no mido** esos RTT. Y en cada retransmisión **duplico** el timeout.
- **Jacobson/Karels:** tener en cuenta también la **variación**. $\text{Diff} = \text{Sample} - \text{EstRTT}$; $\text{EstRTT} += \delta \cdot \text{Diff}$; $\text{Dev} += \delta(|\text{Diff}| - \text{Dev})$; $\text{Timeout} = \text{EstRTT} + 4 \cdot \text{Dev}$. Si el RTT es estable, el timeout queda justo; si varía mucho, se agranda.
- **RFC 6298:** la misma idea formalizada. Primera medición R : $\text{SRTT} = R$, $\text{RTTVAR} = R/2$. Después: $\text{RTTVAR} = \frac{3}{4}\text{RTTVAR} + \frac{1}{4}|\text{SRTT} - R'|$ (primero), $\text{SRTT} = \frac{7}{8}\text{SRTT} + \frac{1}{8}R'$. $\text{RTO} = \text{SRTT} + \max(G, 4 \cdot \text{RTTVAR})$, con mínimo 1 s (y 1 s antes de la primera medición).

9.6 Segmentos chiquitos: Nagle y Silly Window

- Problema: en telnet cada tecla genera 4 segmentos y 162 bytes para 1 byte útil.
- **Algoritmo de Nagle** (lado *emisor*): si tengo un MSS lleno, lo mando. Si no, y tengo datos en vuelo sin confirmar, **espero el ACK** juntando datos. Si no hay nada en vuelo, mando ya. Resultado: a lo sumo un segmento chico por RTT.
- **Silly Window Syndrome** (lado *receptor*): la app lee de a 1 byte, el receptor anuncia ventanas de 1 byte y el emisor manda segmentos de 1 byte. **Solución de Clark:** no anunciar ventanas chicas; esperar a tener libre $\min(\text{MSS}, \text{buffer}/2)$.

9.7 Llenar el caño (otra vez)

Para usar todo el enlace la ventana tiene que ser $\geq \text{RTT} \times \text{ancho de banda}$. Pero **AdvertisedWindow** tiene 16 bits, o sea un máximo de **64 KB**: con RTT de 100 ms alcanza para un T1 (18 KB) y ya no para Ethernet de 10 Mbps (122 KB). **libro** Solución: opción *window scale* (RFC 1323), que multiplica la ventana. Esa RFC también agrega *timestamps* para medir mejor el RTT y distinguir números de secuencia que dan la vuelta en redes rápidas.

Para recordar

Handshake de 3 pasos, cierre de 4, $TIME_WAIT = 2MSL$. Flujo: AdvertisedWindow (receptor) vs congestión (red). RTO adaptativo: Karn (no medir retransmitidos + backoff) y Jacobson (media + 4 desvíos). Nagle es del emisor; Clark (silly window) del receptor. Ventana $\geq RTT \times BW$; 64 KB no alcanza.

Preguntas típicas para autoevaluarte

1. ¿Por qué el ancho de banda limita la velocidad? (Fourier + filtro)
2. Nyquist vs Shannon. ¿Por qué no subir niveles para siempre?
3. ¿Qué mide la entropía? ¿Qué relación tiene con Huffman?
4. Componentes del delay; ¿cuál domina en cada caso?
5. Teorema de muestreo; ¿por qué PCM usa 64 kbps?
6. ¿Por qué Manchester duplica el baud rate? ¿Qué resuelve de NRZ?
7. ¿Por qué S&W es ineficiente? ¿Cómo se dimensiona la ventana? ¿Cuántos números de secuencia hacen falta?
8. ¿Por qué Ethernet tiene trama mínima? ¿Qué pasa si sube la velocidad?
9. CSMA/CA vs CSMA/CD; estación oculta y expuesta.
10. ¿Para qué sirve STP y cómo elige los puertos?
11. Dominio de colisión vs de broadcast. ¿Para qué sirven las VLANs?
12. Datagramas vs circuitos virtuales.
13. Fragmentación IP: dónde se parte, dónde se arma, qué campos intervienen.
14. DV vs LS; conteo a infinito y sus parches.
15. ¿Por qué OSPF es jerárquico y qué se pierde?
16. ¿Por qué hace falta BGP? Tipos de AS.
17. Transporte vs enlace: ¿por qué no alcanza la misma ventana?
18. ¿Por qué el RTO es adaptativo? Karn y Jacobson.
19. Nagle vs Silly Window: ¿de qué lado actúa cada uno?
20. ¿Por qué 16 bits de ventana son un problema? $TIME_WAIT$ y persist timer.